

Chargement, inspection, filtrage

Analyse des données — Séance 2

Stefania Marcassa

CY Cergy Paris Université — L3 Économie

Où nous allons aujourd'hui

1. Charger un vrai fichier, avec ses vrais défauts.
2. Comprendre ce qu'il contient — c'est-à-dire lire sa documentation.

Le fil de la séance

Un fichier de données ne se comprend pas tout seul. Sans sa documentation, un tableau de chiffres n'est qu'un tableau de chiffres.

Les bibliothèques

Ne pas réécrire ce qui existe

Une bibliothèque est du code écrit par d'autres, testé, documenté, et mis à disposition. On l'importe plutôt que de le réécrire.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

L'abréviation après `as` est une convention, pas une obligation — mais c'est une convention universelle. Écrivez `pd`, jamais `pandas` ni `p`.

Ces trois bibliothèques sont déjà installées sur Colab. Il n'y a rien à télécharger.

Quand une bibliothèque manque

```
import statsmodels.api as sm
```

```
ModuleNotFoundError: No module named 'statsmodels'
```

Ce message ne dit pas que vous avez fait une faute de frappe. Il dit que la bibliothèque n'est pas installée.

```
!pip install statsmodels
```

Le ! en début de ligne indique à Colab qu'il s'agit d'une commande système, non de Python. Cela vous servira une fois ou deux dans le semestre.

Charger un fichier

Charger depuis une adresse, pas depuis votre disque

```
url = "https://www.data.gouv.fr/.../emissions.csv"  
df = pd.read_csv(url)
```

C'est la méthode à privilégier systématiquement. Le notebook devient **reproductible** : il s'exécute à l'identique chez votre binôme, chez votre correcteur, et sur votre machine dans six mois.

```
from google.colab import files  
fichiers = files.upload()
```

Le téléversement reste nécessaire pour les fichiers exigeant une inscription — de nombreuses données d'enquête. Il rend alors le notebook dépendant de votre disque, ce qui doit être signalé en commentaire.

Lire, et vérifier ce qu'on a lu

```
df = pd.read_csv("cm02-communes.csv")

df.shape          # (nb de lignes, nb de colonnes)
df.columns        # les noms de variables
df.head()         # les 5 premières lignes
df.info()         # types et valeurs non manquantes
```

Ces quatre lignes se tapent **systématiquement** , avant tout calcul.

Un fichier chargé sans être inspecté est un fichier dont vous ignorez le contenu. La plupart des erreurs de ce cours viendront d'une inspection sautée.

Les fichiers français ne sont pas des fichiers anglais

```
df = pd.read_csv("cm02-communes.csv")           # une seule colonne ?
```

Trois conventions différentes, et il faut les déclarer :

```
df = pd.read_csv(
    "cm02-communes.csv",
    sep=";",          # séparateur point-virgule, pas virgule
    decimal=".",      # virgule décimale
    thousands=" ",    # espace comme séparateur de milliers
    encoding="latin-1" # accents des fichiers anciens
)
```

Le symptôme d'un mauvais séparateur : `shape` annonce une seule colonne. Celui d'un mauvais encodage : `UnicodeDecodeError`, ou des accents remplacés par des symboles.

```
df = pd.read_excel("dares_offres.xlsx",  
                  sheet_name="Donnees",  
                  skiprows=4)
```

Les tableaux publiés par les administrations comportent presque toujours un titre, une source et des notes autour des données. `skiprows` saute les lignes d'en-tête ; il faut d'abord **ouvrir le fichier et compter**.

Le réflexe

Avant de charger un fichier inconnu, ouvrez-le dans un tableur et regardez à quoi il ressemble. Trente secondes qui en économisent vingt minutes.

Comprendre ce qu'on a chargé

Un fichier d'enquête est accompagné d'un document qui décrit chaque variable : son nom, sa signification, ses modalités, son mode de collecte.

Sans ce document, le fichier est illisible. Avec lui, tout devient mécanique.

La compétence la plus sous-estimée du cours

Savoir chercher dans un dictionnaire des variables. Ce n'est pas une compétence informatique, et c'est celle qui distingue le plus nettement les étudiants en M1.

À quoi ressemble une entrée

Champ	Contenu
Nom	Code court, souvent obscur
Libellé	La question posée, ou le concept mesuré
Type	Quantitative, ou qualitative à modalités
Modalités	1 = ..., 2 = ..., 3 = ...
Champ	Sur qui la variable est-elle renseignée ?
Non-réponse	Comment est-elle codée ?

Les deux dernières lignes sont celles qu'on oublie de lire, et celles qui font les résultats faux.

Deux pièges de modalités

Le champ de la variable

Dans notre fichier, le taux de chômage n'est pas diffusé pour les communes de moins de 2 000 habitants — secret statistique. L'absence n'est donc pas aléatoire : elle vise les petites communes.

La non-réponse codée en chiffres

Le nombre d'établissements vaut 9 999 quand il n'est pas renseigné. La moyenne se calcule sans broncher, et elle est fausse.

Il faut convertir ces codes en valeurs manquantes — c'est le sujet de la séance 3.

Aucun de ces deux problèmes n'est détectable sans la documentation.

Pour la suite

Vous partirez des fichiers détail du recensement, en accès libre, et vous reconstruirez le statut d'activité : actif occupé, chômeur, inactif.

- Retrouver dans la documentation les variables qui définissent ce statut
- Reconstruire les trois catégories
- Calculer un taux de chômage
- Le comparer au taux BIT publié, et expliquer l'écart

L'écart existera, et il sera de plusieurs points. Il ne viendra pas d'une erreur de code, mais de la définition : le recensement enregistre un chômage **déclaré**, là où le BIT exige trois critères vérifiés.

1. Terminez le chargement et l'inspection du fichier du TD.
2. Exécutez les trois vérifications sur vos données.
3. Repérez, sans les traiter, les variables qui contiennent des valeurs manquantes.
4. Récupérez le notebook de la séance 3 sur le site du cours.

`stefaniamarcassa.github.io/analyse_des_donnees`

Des questions ?